

Transacciones



Universidad de Oviedo

Este material está adaptado principalmente del curso *CS145 Introduction to Databases* impartido por **Christopher Re** en **Universidad de Stanford** (<https://cs145-fa18.github.io/>)

Escuela de Ingeniería Informática

2026-2027

Introducción

Propiedades de las transacciones

Concurrencia, Planificación y Serialización

Anomalías que pueden ocurrir

¿Cómo garantizar que no se producirán anomalías?

Gestión de transacciones JDBC

You may find Jennifer Widom videos on transactions useful:
<https://www.youtube.com/watch?v=-NPyRXCysW0>
<https://www.youtube.com/watch?v=usgUg08xNDY>
<https://www.youtube.com/watch?v=zz-Xbqp0g0A>

Section 1

Introducción

Almacenamiento y rendimiento¹

Motivación



Srīgi
@srīgi

...

"Latency Numbers Every Programmer Should Know"

It is hard for humans to get the picture until you translate it to "human numbers":

[Traducir post](#)

1 CPU cycle	0.3 ns	1 s
Level 1 cache access	0.9 ns	3 s
Level 2 cache access	2.8 ns	9 s
Level 3 cache access	12.9 ns	43 s
Main memory access	120 ns	6 min
Solid-state disk I/O	50-150 µs	2-6 days
Rotational disk I/O	1-10 ms	1-12 months
Internet: SF to NYC	40 ms	4 years
Internet: SF to UK	81 ms	8 years
Internet: SF to Australia	183 ms	19 years
OS virtualization reboot	4 s	423 years
SCSI command time-out	30 s	3000 years
Hardware virtualization reboot	40 s	4000 years
Physical system reboot	5 m	32 millenia

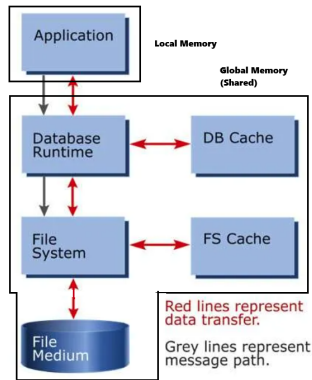
- **Memoria principal: Acceso rápido, capacidad limitada y volátil.**
- **Disco: Acceso lento, gran capacidad y persistente.**
- **Mejorar el rendimiento utilizando ambas eficazmente.**

¹Posteado en X por Peter Norvig

Organización de la memoria en niveles

Cómo los utiliza un SGBD

1. **Local:** Cada proceso tiene su propia área de memoria local de acceso privado.
2. **Global o compartida:** Los procesos pueden leer de o escribir en memoria global compartida.
3. **Disco:** La memoria global puede ser leída desde o volcada (flush) a disco.
4. **Log**



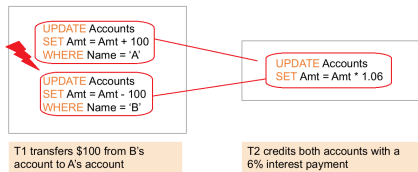
Message path and data transfers in a persistent database system.

<https://www.mcobject.com/>

[what-is-a-persistent-database-and-how-is-it-different/](#)

Motivación. Ejemplo

Transferir dinero entre cuentas. Pagar intereses



Requisitos:

- Resistencia a fallos del sistema, como fallos de hardware o caídas del sistema.
- Consistencia de datos
- Durabilidad. Si la operación tiene éxito, la nueva información perdurará en el tiempo.

Otras lecturas [1]

Transacciones: Solución a los problemas de concurrencia y fallos

Una **transacción** (*txn*) es una **secuencia** de una o más **operaciones SQL** que **se ejecutan como una única unidad lógica de trabajo**.

Las transacciones **comienzan implícitamente**

- Al establecer una conexión
- Cuando acaba la txn en curso

Una transacción **finaliza**:

- Implícitamente, al cerrarse la conexión (o por fallo del sistema).
- Con **Commit**, que hace permanentes los cambios y abre nueva txn.
- Con **Rollback**, que descarta los cambios y abre una nueva txn.

T1: START TRANSACTION

UPDATE Accounts
SET Amt = Amt + 100
WHERE Name = 'A'

UPDATE Accounts
SET Amt = Amt - 100
WHERE Name = 'B'

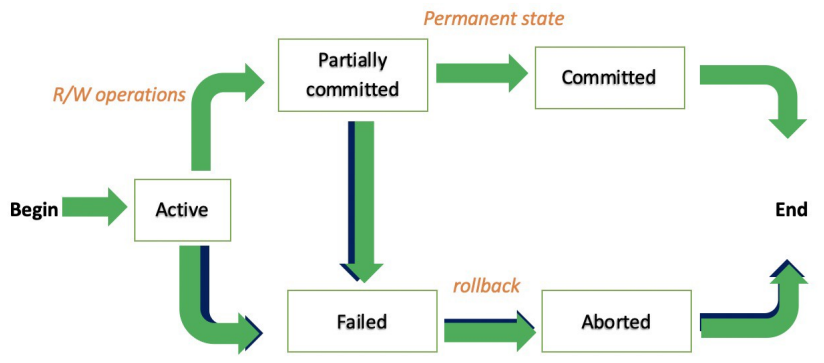
COMMIT

T2: START TRANSACTION

UPDATE Accounts
SET Amt = Amt * 1.06

COMMIT

Ciclo de vida



Section 2

Propiedades de las transacciones

ACID

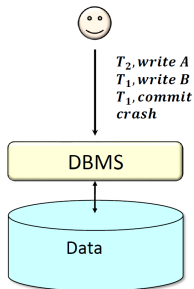
Guía rápida del comportamiento esperado de las transacciones



ACID: Atomicidad

Una vez que se finaliza una transacción, el contenido de la base de datos es como si

- Todas las operaciones de la transacción se hubieran completado con éxito, o
- No se hubiera ejecutado ninguna (en caso de fallo).



El sistema necesita **DESHACER T2** ya que **NO había comprometido (commit) los datos** en el momento del fallo del sistema.

ACID: Consistencia

Las transacciones deben conducir la BBDD de un estado consistente a otro estado consistente. Es **responsabilidad**:

- del SGBD, número de cuenta es único, saldos no negativos, ...
- y de la lógica de negocio para reglas que la base de datos no puede imponer (ej. que la cantidad de dinero no cambie).

Total before T occurs = 500

Total after T occurs = 500

T
Read (X) $X := X - 50$ Write (X) Read (Y) $Y := Y + 50$ Write (Y)

- **Durante**: se pueden ver datos **temporalmente** inconsistentes
- **commit**: sólo se salvan datos que cumplan las restricciones.

ACID: Aislamiento

Aunque varias txn se ejecuten simultáneamente, no interfieren y el efecto final debe ser el mismo que si **cada una se ejecutase sólo**

- O se ejecuta **completamente** una y luego la otra.
- O se **intercalan** sin que se produzca resultados incorrectos.

Los cambios hechos por una no son visibles por otras hasta el commit.

X = 500 Rs		Y = 500 Rs	
T		T''	
Read (X)		Read (X)	
X := X - 50		Read (Y)	
Write (X)		Z := X + Y	
Read (Y)		Write (Z)	
Y := Y + 50			
Write (Y)			

ACID: Durability

- Una vez que una transacción ha sido **confirmada (commit)**, sus efectos deben ser permanentes, incluso si después falla el sistema.
- Mientras una transacción está en curso (**parcialmente confirmada**), los efectos no son persistentes.
- Si algo falla, se restaurará la BBDD a un estado consistente antes de que comience la transacción.

Bloque 1 cuestionario

Section 3

Concurrencia, Planificación y Serialización

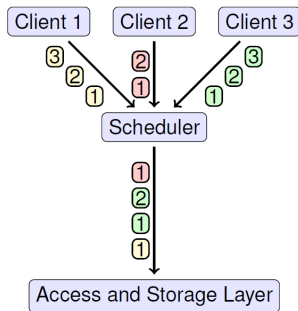
Operaciones SQL como lecturas y escrituras

Las operaciones SQL pueden verse como una lista de lecturas (**R(O)**) y escrituras (**W(O)**) sobre objetos O

Ejemplo: $T_1 : R(V), R(Y), W(V), W(C)$

SQL	Lecturas y Escrituras implícitas
SELECT X FROM ...	$R(X)$
INSERT INTO tabla (X) VALUES (...)	$W(X)$ (crea un nuevo registro)
UPDATE tabla SET X = ... WHERE ...	$R(X), W(X)$ (lectura (localiza registro) + escritura (actualizar))
DELETE FROM tabla WHERE ...	$R(X), W(X)$ (lectura (localiza registro) + escritura (eliminar))

Planificación de transacciones



- Una **planificación** es el intercalado de operaciones de un conjunto de txn.
- En una planificación el orden de dos operaciones de una txn T debe ser en el mismo en que aparecen en T .

$T_1 : R(V) W(V)$

$T_2 : R(Y) W(Y)$

Which of the following is a schedule of these transactions?

$S_1 :$	T_1	$R(V)$	$W(V)$
	T_2	$R(Y)$	$W(Y)$
$S_2 :$	T_1	$W(V)$	$R(V)$
	T_2	$R(Y)$	$W(Y)$

Planificación serie y planificación serializable

Una planificación es **serie** si las operaciones de las transacciones no se intercalan sino que se ejecutan una tras otra.

Una planificación es **serializable** si su efecto en la BBDD es el mismo que alguna planificación serie.

- ¿ Hay alguna planificación serie en la imagen? ¿ Y serializable?

T_1	R(V)	W(V)
T_2	R(Y)	W(Y)

T_1	R(V)	W(V)
T_2	R(Y)	W(Y)

- Es deseable una planificación serializable frente a una planificación serie, ¿ por qué?
- No se permiten planificaciones no serializables, ¿ por qué?

Planificación serie

T1: START TRANSACTION

UPDATE Accounts
SET Amt = Amt + 100
WHERE Name = 'A'

UPDATE Accounts
SET Amt = Amt - 100
WHERE Name = 'B'

COMMIT

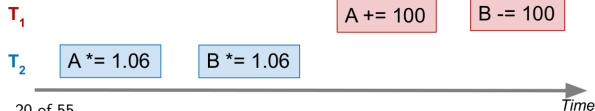
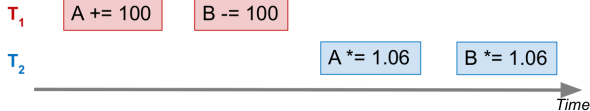
T2: START TRANSACTION

UPDATE Accounts
SET Amt = Amt * 1.06

COMMIT

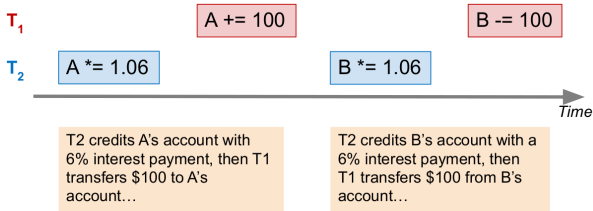
T1 transfers \$100 from B's account to A's account

T2 credits both accounts with a 6% interest payment

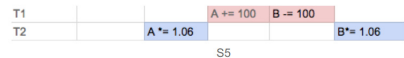
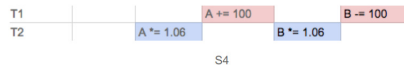
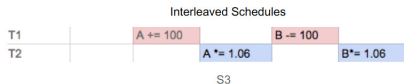
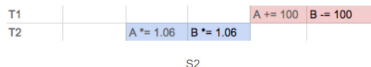
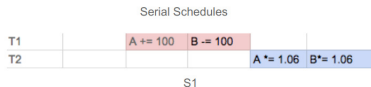


Planificación serializable

El SGBD debe garantizar que con la **planificación** (intercalado) propuesta se garantizan las propiedades **ACID**



Ejemplos de planificación



Planificación serie	S_1, S_2
Planificación serializable	S_3, S_4
Planificación equivalente	$\langle S_1, S_3 \rangle \langle S_2, S_4 \rangle$
Planificación no serializable (:)	$\langle S_5, S_6 \rangle$

Bloque 2 cuestionario

Grafo de precedencia

Es un **grafo dirigido** que representa las dependencias entre transacciones a partir de los conflictos entre sus operaciones de lectura y escritura sobre los mismos datos.

- **Nodos:** cada nodo representa una transacción.
- **Aristas dirigidas:** existe arista $T_i \rightarrow T_j$ si una operación de T_i precede y entra en conflicto con una de T_j .

Dos operaciones entran en **conflicto** si:

- Acceden al mismo objeto.
- Al menos una es una escritura.
- Son de diferentes transacciones.

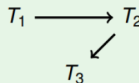
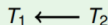
Comprobar si una planificación es serializable

Una planificación es **serializable** si **no hay ciclos** en el grafo.

T_1	W(V)	W(V)
T_2	R(V)	

T_1	R(V)	W(V)
T_2	R(V)	

T_1		W(Y)
T_2	R(V)	R(Y) W(Z)
T_3	W(V)	



Schedules 2 and 3 have no cycles in their precedence graph.
They are conflict serializable!

Ejercicio

Considera txn T_1 , T_2 y T_3 y dos planificaciones posibles

Planificación 1	Planificación 2
$R_1(X)$	$R_1(X)$
$R_3(X)$	$R_3(X)$
$W_1(X)$	$W_3(X)$
$R_2(X)$	$W_1(X)$
$W_3(X)$	$R_2(X)$

Determina qué planificación, si la hay, es serializable.

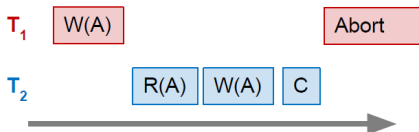
Bloque ejercicios 3

Section 4

Anomalías que pueden ocurrir

Dirty read / Lectura de datos no comprometidos

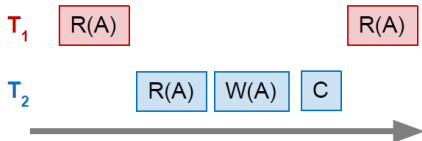
Una txn lee datos que otra ha modificada pero no comprometido aún.



1. T_1 **escribe** un valor en A
2. T_2 **lee** valor de A, escribe en A, **commit**
3. T_1 **aborta** - en este momento, el resultado producido por T_2 está basado en un valor obsoleto / inconsistente

Unrepeatable read, Lectura no repetible

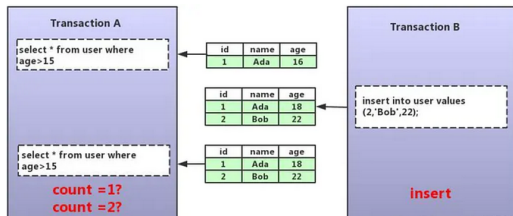
Una txn lee un dato dos veces y recibe valores diferentes, debido a una modificación confirmada por otra transacción entre ambas lecturas



1. T_1 **lee** el valor de A
2. T_2 **escribe** el valor en A
3. T_1 lee el valor A otra vez y obtiene un valor diferente

Phantom Read, Lectura fantasma

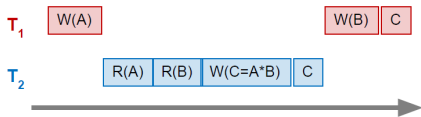
Cambia el número o identidad de filas devueltas por una consulta repetida, porque otra txn agrega, elimina o modifica filas que cumplen la condición.



Lectura de datos parcialmente comprometidos

Reading partial commits

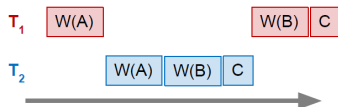
Ocurre cuando una transacción **lee solo parte de los cambios** realizados por otra antes de que esta haga *COMMIT*.



1. T_1 **escribe** un valor en A
2. T_2 **lee** A y B, escribe un valor que depende de A y B
3. T_1 escribe en B - ahora el resultado de T_2 se basa en valores inconsistentes

[Partially] lost update, Pérdida [parcial] de la actualización

Dos txn concurrentes modifican los mismos datos y la escritura de una sobrescribe los cambios de la otra.



1. T_1 **escribe** valor en A (actualiza)
2. T_2 **escribe** (actualiza) en A y B
3. T_1 luego escribe en B; ahora el valor de T_1 es el de B y el de T_2 el de A - no equivalente a ninguna planificación serie!

Ejercicios

		T_1	T_2	
Supongamos que el valor inicial de A y B es 1000.	i_{11}	$x = R(A)$	$y = R(A)$	i_{21}
	i_{12}	$x = x - 50$	$temp = y * 0.1$	i_{22}
Supón que la T_1 transfiere 50 de A a B y la transacción T_2 retira el 10% del valor de A.	i_{13}	$W(A=x)$	$aux = y - temp$	i_{23}
	i_{14}	$R(y=B)$	$W(A=aux)$	i_{24}
	i_{15}	$y = y + 50$	COMMIT	i_{25}
	i_{16}	$W(B=y)$		
	i_{17}	COMMIT		

Calcula los valores finales si T_1 y T_2 se ejecutan en serie.

Ahora, considera esta planificación $i_{11}, i_{12}, i_{21}, i_{22}, i_{23}, i_{13}, i_{14}, i_{24}, i_{25}, i_{15}, i_{16}, i_{17}$. Identificar la anomalía.

Supón que el valor inicial de A y B es 1000; la transacción T_1 transfiere 50 de la cuenta A a la B, y la transacción T_2 calcula el 10% de interés del saldo de A y lo suma al valor de A.		T_1		T_2	
	i_{11}	R(A)		R(A)	i_{21}
	i_{12}	A=A-50		aux=A * 0.1	i_{22}
	i_{13}	W(A)		A=A+aux	i_{23}
	i_{14}	R(B)		W(A)	i_{24}
	i_{15}	B=B+50		COMMIT	i_{25}
	i_{16}	W(B)			
	i_{17}	COMMIT			

Ahora, considera la planificación $i_{11}, i_{12}, i_{13}, i_{21}, i_{22}, i_{23}, i_{24}, i_{25}$. T_1 decide hacer **rollback** por algún motivo. Identificar la anomalía.

Section 5

¿Cómo garantizar que no se producirán anomalías?

Las transacciones requieren **mecanismos de control de concurrencia**, que garanticen la corrección incluso cuando se ejecutan de forma intercalada.

Control de concurrencia basado en bloqueos

Enfoque pesimista

Evitar conflictos empleando un mecanismo de **bloqueo, pesimista** o **Lock-based Concurrency Control**

- Cada lectura requiere un bloqueo compartido (**shared lock**)
 - Bloquea escritores pero permite lectores concurrentes.
 - Se libera explícitamente o al final de la txn
- Cada escritura requiere un bloqueo exclusivo (**exclusive lock**).
 - No permite ni lecturas ni escrituras concurrentes.
 - Se libera al final de la txn

Sin embargo, el bloqueo genera **competencia** por los recursos, y la competencia afecta la **escalabilidad**.

Granularidad.

Schedule with explicit lock actions

T_1	$X(B)$	$W(B)$	$U(B)$			
T_2	$S(A)$	$R(A)$	$X(B)$	$W(B)$	$U(A)$	$U(B)$

Here we use the following abbreviations:

- $S(A)$ = shared lock on A
- $X(A)$ = exclusive lock on A
- $U(A)$ = unlock A , or if more precision is needed
 - $US(A)$ = unlock shared lock on A
 - $UX(A)$ = unlock exclusive lock on A

Control de concurrencia basado en versiones (**MVCC**)

Enfoque optimista

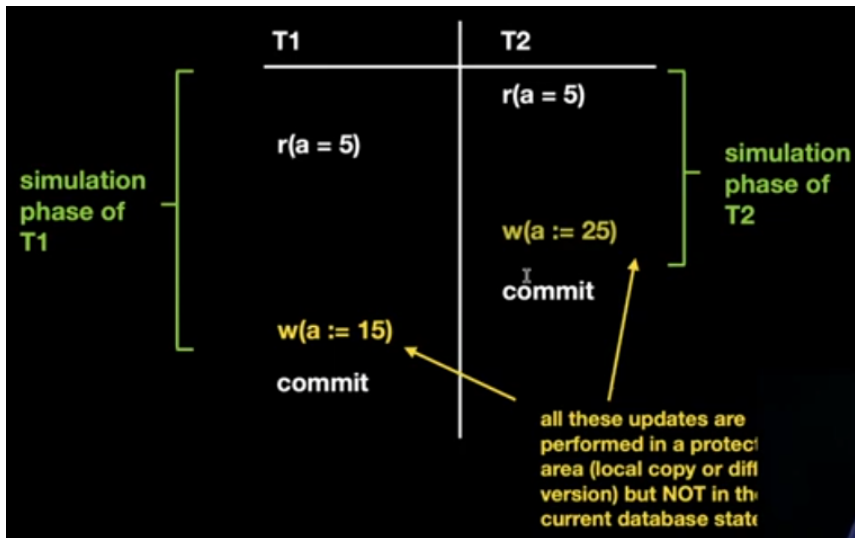
- **Asume** que **no habrá conflictos**
- Las transacciones leen y escriben **sin bloquearse**
- Al **commit**, se **valida** que los datos leídos y modificados sigan siendo válidos.

Las transacciones proceden en tres fases

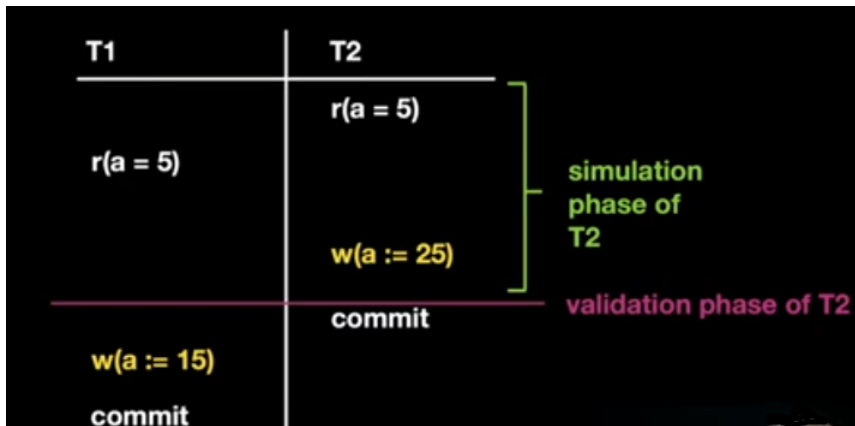
1. **Simulación**: lecturas sobre versiones consistentes y escrituras en un espacio privado.
2. **Validación**: se comprueba la ausencia de conflictos; si los hay, la transacción aborta.
3. **Escritura**: los cambios se hacen visibles en el estado global; la persistencia en disco puede ser posterior.

Ver vídeo <https://www.youtube.com/watch?v=SGR-WR4w-Jk>

Primera fase: Fase de simulación



Segunda fase: Fase de validación, chequeo de conflictos



Segunda fase: Read/Write Sets

	T1	T2	
		<code>r(a = 5)</code>	<code>read_set_T2 = {a}</code>
<code>read_set_T1 = {a}</code>	<code>r(a = 5)</code>		
<code>write_set_T1 = {}</code>		<code>w(a := 25)</code>	<code>write_set_T2 = {a}</code>
		<code>commit</code>	validation phase of T2
<code>write_set_T1 = {a}</code>	<code>w(a := 15)</code>		
	<code>commit</code>		

Segunda fase: Intersección Write Sets

	T1	T2	
		r(a = 5)	read_set_T2 = {a}
read_set_T1 = {a}	r(a = 5)		
write_set_T1 = {}		w(a := 25)	write_set_T2 = {a}
		commit	validation phase of T2
write_set_T1 = {a}	w(a := 15) commit		

Did any other transaction commit item 'a' while T2 was running?

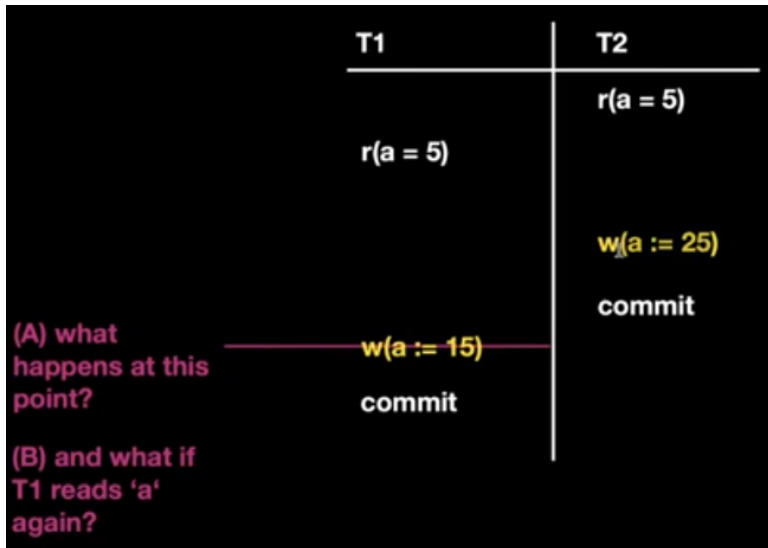
=> No other TA committed while T2 was running.

=> proceed to Commit Phase

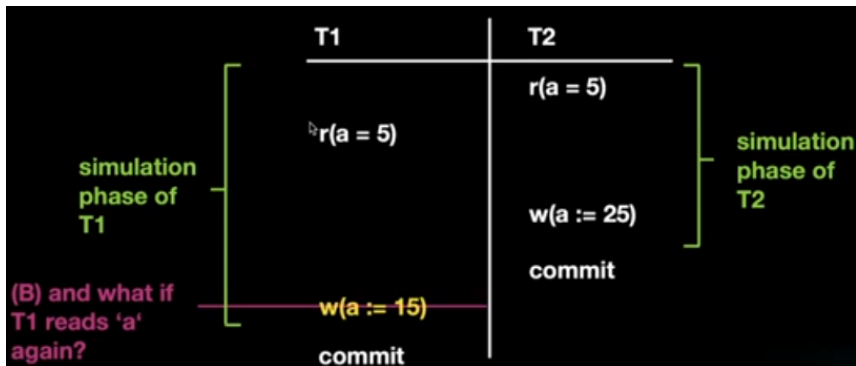
Tercera fase: Commit, aplicar cambios

T1	T2
r(a = 5)	r(a = 5)
	w(a := 25)
	commit
w(a := 15)	commit phase of T2
commit	local version of T2 becomes the current global version, in this case we only set data item a := 25

T1 continúa ...

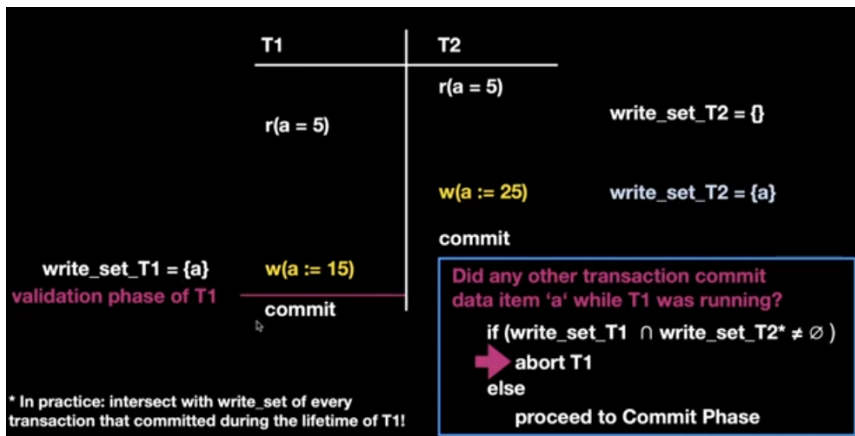


Primera fase: Fase de simulación



(B) T1 will not read the more recent version of 'a' as T1 operates on a snapshot of the data as of the time T1 started: This is why we call it **SNAPSHOT ISOLATION**. However: isolation level **READ COMMITTED** would allow for this!

Write-sets intersection: $T1 \cap \text{others txn}$: Fallo



Niveles de aislamiento

La aplicación no trabaja con bloqueos ni vistas de la BBDD.

Puede proporcionar al SGBD **sugerencias** sobre cómo gestionar la concurrencia, indicando al SGBD el grado de **aislamiento** deseado.

Un **nivel de aislamiento** define qué efectos de unas transacciones pueden ser **observados** por otras transacciones concurrentes.

En sistemas basados en bloqueos, restringiendo las operaciones concurrentes.

En MVCC, determinando qué versiones de los datos son visibles para cada transacción.

- Por transacción
- El nivel de aislamiento afecta a las operaciones de lectura:
determina qué datos puede observar una transacción durante su ejecución.
- El nivel de aislamiento de una transacción no afectará a otras txn concurrentes.

Weaker "Isolation Levels"

Read Uncommitted
Read Committed
Repeatable Read

Strongest "Isolation Levels"

Serilizable order

↓ Overhead ↑ Concurrency

↓ Consistency Guarantees

1. **Lectura no confirmada (menos restrictiva)**

- Se pueden leer datos no confirmados.
- Una lectura puede devolver cambios que luego sean deshechos.

2. **Read Committed**

- No permite leer datos no confirmados.
- Lecturas sucesivas del mismo dato pueden devolver valores diferentes, siempre confirmados.

3. **Repeatable Read**

- Lecturas repetidas de los mismos registros devuelven los mismos valores.
- Sin embargo, pueden aparecer nuevos registros en lecturas sucesivas.

4. **Serializable (más restrictivo)**

- Las transacciones concurrentes producen el mismo resultado que si se ejecutaran de forma serie.

Anomalías vs Niveles aislamiento

Isolation levels ↓ Read phenomena →	Dirty reads	Non-repeatable reads	Phantom reads
Read uncommitted	YES	YES	YES
Read committed	NO	YES	YES
Repeatable read	NO	NO	YES
Serializable	NO	NO	NO

Ejercicios

Sea la tabla $W(\underline{\text{name}}, \text{pay})$ y dos txns concurrentes. Supón que $\text{pay}=50$ antes de que se ejecute ninguna instrucción. Recuerda, las instrucciones SQL individuales **siempre se ejecutan atómicamente**.

T_1 :

Begin Transaction

S1: update W set pay = 2*pay where name = 'Amy'

S2: update W set pay = 3*pay where name = 'Amy'

Commit

T_2 :

Begin Transaction

S3: update W set pay = pay-20 where name = 'Amy'

S4: update W set pay = pay-10 where name = 'Amy'

Commit

Supón que T_1 y T_2 se ejecutan hasta el final, con diferentes niveles de aislamiento. ¿Cuáles son los valores posibles de pay si

1. ambos se ejecutan con Serializable?
2. ambos se ejecutan con Read-Committed?.
3. T_1 es read-committed y T_2 es read-uncommitted?.
4. ambos se ejecutan con Read-Uncommitted?.
5. ambos se ejecutan con nivel de aislamiento Serializable. T_1 se ejecuta hasta el final, pero T_2 aborta después de la instrucción S3 y no se vuelve a ejecutar.

Section 6

Gestión de transacciones JDBC

Las transacciones **comienzan implícitamente**

- Al establecer una conexión
- Cuando acaba la txn en curso

Una transacción **finaliza**:

- al cerrarse la conexión (también por fallo en el sistema).
- **Commit** hace permanentes los cambios realizados.
- **Rollback** descarta los cambios realizados.

La propiedad AutoCommit en la conexión

- `true` (valor por defecto): cada sentencia SQL se confirma automáticamente tras su ejecución.
- `false`: las sentencias forman parte de la transacción en curso y requieren confirmación explícita con `con.commit()` o anulación con `con.rollback()`.

- Para verificar el modo de confirmación,

```
public boolean getAutoCommit()
```

- Para cambiarlo,

```
void setAutoCommit(boolean b)
```

- Para comprobar si la conexión está todavía abierta

```
public boolean isClosed()
```

- Para establecer o consultar el **nivel de aislamiento** para la conexión actual

```
public int getTransactionIsolation() and
```

```
void setTransactionIsolation(int level)
```

Lecturas adicionales (muy recomendables) (1)

- [1] <https://www.codemag.com/Article/0607081/Database-Concurrency-Conflicts-in-the-Real-World>
- [2] <https://www.youtube.com/watch?v=FA85kHsJss4> and <https://www.youtube.com/watch?v=aNCpEC0-VVs>
- [3] <https://cs.stanford.edu/~chrismre/>
- [4] <https://cs.stanford.edu/people/widom/>
- [5] <https://es.slideshare.net/brshristov/database-transactions-and-sql-server-concurrency>
- [6] <https://ucbrise.github.io/cs262a-spring2018/notes/07-concurrency.pdf>
- [7] <https://15445.courses.cs.cmu.edu/fall2018/>

Lecturas adicionales (muy recomendables) (2)

[8] <https://www.youtube.com/watch?v=U3SHusK80q0>